GERENCIAMENTO DE SEGREDOS: CHAVES SSH E GPG

*Um robô que varre o GitHub achou, num commit de dois anos antes, uma `deploy_key` privada sem senha. O commit fora revertido no dia seguinte — mas **reverter não apaga o objeto do histórico**, e a chave ficou recuperável por 730 dias. Ninguém rotacionou nada porque ninguém sabia que havia algo a rotacionar.*

086
GUIAS LINUX

POR QUE A CHAVE VENCE A SENHA: A PRIVADA ASSINA, E NUNCA TRAFEGA

CLIENTE `~/.ssh/id_ed25519`
a chave **PRIVADA**, protegida por senha — nunca sai desta máquina

SERVIDOR `~/.ssh/authorized_keys`
guarda **SÓ A PÚBLICA**; comprometê-lo não entrega sua credencial

1	O cliente anuncia **qual** chave pública pretende usar.	cliente →
2	O servidor confere se ela consta do **`authorized_keys`**.	← servidor
3	O servidor devolve um **desafio aleatório**, novo a cada conexão.	← servidor
4	O cliente **assina o desafio** com a privada — e envia a assinatura, não a chave.	cliente →
5	O servidor verifica a assinatura com a pública. **Sessão aberta.**	servidor

Tudo muda no passo 4. A senha precisa ser *entregue* para ser verificada: quem verifica passa a conhecê-la, ela trafega e é reutilizável em outro serviço. A privada só **assina**, e é a assinatura que viaja — o servidor nunca vê o segredo, e um desafio capturado não serve para a conexão seguinte.

DOIS FORMATOS, DOIS PROBLEMAS DIFERENTES

SSH — AUTENTICAR UMA SESSÃO

"sou eu que estou conectando, agora"

- **`ssh-keygen -t ed25519 -C "vitor@estacao"`** é o padrão hoje: curta, rápida, sem parâmetro para errar. RSA 4096 só para servidor velho; **DSA nunca**.
- **Senha na chave não é opcional** — sem ela o arquivo *é* o acesso. O **`ssh-agent`** pede uma vez por sessão; **`ssh-add -t 3600`** faz ele esquecer.
- **Chave de automação vem restrita:** **`restrict`** desliga tudo, **`command=`** fixa o programa, **`from=`** fixa a origem. Mesmo vazada, faz uma coisa só.
- **O comentário `-C` é para o futuro:** um dia você olha um **`authorized_keys`** de nove linhas e precisa decidir quais apagar.

GPG — PROVAR PROCEDÊNCIA DO DADO

"este arquivo é o que o projeto publicou"

- **Primária × subchaves:** a primária (**`sec`**) é a identidade e fica offline; as subchaves (**`ssb`**) trabalham e são revogáveis uma a uma — uma comprometida não derruba sua rede de confiança.
- **O primeiro comando depois de gerar a chave** é **`gpg --gen-revoke`**: o certificado de revogação não pode ser criado *depois* que a chave se perdeu.
- **`Good signature` prova que o dado não mudou** — não que a chave é de quem você pensa. O aviso *"not certified"* é o passo que falta.
- **Confira a impressão digital por canal independente.** Chave e assinatura vindas do mesmo servidor protegem contra corrupção, não contra adversário.

NA PRÁTICA — E ONDE O SEGREDO FICA GUARDADO

kali@kali: ~

```
$ ssh-keygen -t ed25519 -C "vitor@estacao-2026"
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub vitor@srv
# ~/.ssh 700 e authorized_keys 600, ou o servidor IGNORA
# em silêncio: a causa nº 1 de "ainda pede senha"
$ ssh -v vitor@srv 2>&1 | grep -i 'offering\|accepted'
$ ssh-add -t 3600 ~/.ssh/id_ed25519 # esquece em 1h
# ~/.ssh/config — resolve metade dos problemas
Host interno
    ProxyJump bastion      # salta sem expor agente
    IdentityFile ~/.ssh/id_infra
    IdentitiesOnly yes     # não oferece TODAS as chaves
# authorized_keys: credencial de menor privilégio
restrict,command="/usr/local/bin/receber",
from="10.0.5.0/24" ssh-ed25519 AAAAC3Nza... deploy@ci
$ gpg --armor --detach-sign pacote.tar.gz
$ gpg --verify pacote.tar.gz.asc pacote.tar.gz
```

ONDE GUARDAR	O QUE ISSO CUSTA
variável de ambiente	vaza em `/proc/PID/environ` e em `ps e`
arquivo `0600` no disco	texto claro se o disco não for cifrado
`pass` (GPG + Git)	versionável: o conteúdo já é cifrado
cofre de rede (Vault)	token curto, rotação e auditoria — mas é infra
dentro do repositório Git	**nunca, em hipótese alguma**

Segredo commitado é segredo público — reverter não apaga o objeto, alcançável pelo hash e em todo clone. Rotacione **primeiro**; limpar o histórico vem depois, e o **`gitleaks`** vem antes.

`ForwardAgent yes` entrega suas chaves ao `root` da máquina do meio. quem controla o servidor intermediário usa o socket do seu agente enquanto a sessão está aberta e autentica-se **como você** em todo servidor que aceita sua chave. Para saltar por um *bastion*, use `ProxyJump` (`ssh -J bastion vitor@interno`): a conexão vai de ponta a ponta a partir da sua máquina e nenhum agente fica exposto no caminho.

RESUMO RÁPIDO

- ✓ **A privada assina um desafio e nunca sai da sua máquina** — comprometer o servidor não entrega a credencial. É a diferença estrutural para a senha.
- ✓ **`ed25519` , com senha e com comentário identificável.** Chave sem senha é acesso imediato para quem copiar o arquivo.
- ✓ **`~/.ssh` em 700, `authorized_keys` em 600** — fora disso o servidor ignora o arquivo sem dizer nada.
- ✓ **`ForwardAgent` não; `ProxyJump` sim.** E `IdentitiesOnly yes` evita o "too many authentication failures".
- ✓ **`Good signature` ≠ chave confiável** — falta conferir a impressão digital por canal independente.
- ✓ **Gere o certificado de revogação no minuto seguinte** ao de gerar a chave GPG — depois de perdê-la, ele já não pode ser criado.

NO KALI

Verificar a ISO do Kali é o exercício de GPG mais útil que existe, porque é real: baixe `SHA256SUMS` e `SHA256SUMS.gpg` , importe a chave do projeto, **confira a impressão digital contra o site oficial por HTTPS** e só então verifique a assinatura das somas. Cada elo omitido é uma confiança assumida sem perceber — espelhos são operados por terceiros. **A chave do repositório mantém o `apt` honesto:** todo `apt update` confere a assinatura do `Release` contra o `kali-archive-keyring` ; quando ela expira (`NO_PUBKEY` , `EXPKEYSIG`) a tentação universal é `--allow-unauthenticated` ou `[trusted=yes]` , e isso **desliga a única proteção contra um espelho hostil** — reinstale o chaveiro da fonte oficial. **Senha fraca de chave não resiste às *wordlists*:** o formato antigo do OpenSSH derivava a chave com pouquíssimas iterações, e `ssh-keygen -p -o -a 100 -f ~/.ssh/id_rsa` reencripta sem trocar o par — vale em toda chave anterior a 2018. **A máquina Kali acumula credencial de terceiro** — capturas, *hashes*, chaves de laboratório —, o que a torna alvo de valor desproporcional: disco cifrado e senha na chave GPG são requisitos práticos, não zelo. Material de teste autorizado tem cláusula de custódia; perdê-lo é incidente contratual. **Chave de laboratório não entra em produção**, nem o contrário: pares por contexto (`id_lab` , `id_infra`) com `IdentitiesOnly yes` eliminam a categoria inteira de erro. **RHEL:** mesmos comandos, mas o SELinux exige o contexto `ssh_home_t` — `authorized_keys` copiado de outro lugar é ignorado com permissões perfeitas; `restorecon -Rv ~/.ssh` conserta.